

Nivelación · Fastbook revisado

INTRODUCCIÓN A LA MAQUETACIÓN II

CSS: selectores, modelo de caja, variables y layout

CSS · SELECTORES · BOX MODEL · FLEXBOX · GRID

Apuntes revisados y ampliados

Documento mejorado a partir de las notas originales — a partir de la sección de Selectores

Índice

1. Selectores

- 1.1. Tipos de selectores
- 1.2. Especificidad y cascada
- 1.3. Propiedades

2. Modelo de caja

- 2.1. Visualización externa: block e inline
- 2.2. Las capas de la caja
- 2.3. Box-sizing: estándar vs. alternativo

3. Variables

- 3.1. Sintaxis y uso
- 3.2. Ámbito (scope)
- 3.3. Valores de reserva (fallback)
- 3.4. Importación con @import

4. Dimensiones

- 4.1. Px, em, rem, % y vw/vh

5. Posicionamiento

- 5.1. Static, relative, absolute, fixed y sticky
- 5.2. Z-index

6. Layout

- 6.1. Float
- 6.2. Flexbox
- 6.3. Grid

7. ¿Qué hemos aprendido?

1. Selectores

Todo estilo en CSS se tiene que asignar a un **elemento HTML** a través de un selector; incluso se puede asignar al propio elemento raíz `html`:

```
html {
  font-size: var(--font-size-global);
  color: var(--white-color);
  font-family: "VR_Standard", sans-serif;
}
```

1.1. Especificidad y cascada

Las reglas de estilo se van definiendo siempre en forma de **cascada**: si hay duplicidad entre reglas, se aplica el último estilo definido, o bien el estilo cuyo selector sea **más específico**. Un ejemplo habitual:

```
section {
  color: blue;
}

section {
  font-size: 2rem;
}

section.my-class {
  font-size: 1rem;
}
```

El primer y segundo bloque definen propiedades **distintas** (color y font-size), por lo que ambas se aplican sin conflicto. El tercer bloque sí compite con el segundo: al llevar la clase `.my-class` añadida, el selector es más específico, así que cuando el `section` tenga esa clase, se aplicará `font-size: 1rem` en lugar de `2rem`.

i Ojo con el espacio: `section.my-class` (sin espacio) selecciona un **section que tiene** la clase `my-class`. Pero `section .my-class` (con espacio) selecciona un **elemento hijo** de `section` que tenga esa clase — son selectores completamente distintos. Para evitar confusiones, es más claro escribir `section *.my-class`.

1.1. Tipos de selectores

Más allá de los selectores de elemento, clase o ID, CSS ofrece varias familias de selectores adicionales:

Tipo	Ejemplo	Qué selecciona
------	---------	----------------

Selector de atributo	<code>a[title]</code>	Elementos que tienen un atributo concreto definido
Pseudoclase	<code>button:hover</code> , <code>li:first-child</code>	Un elemento en un estado o posición concretos
Pseudoelemento	<code>button:before</code>	Una parte generada del elemento (no existe en el HTML)
Combinador	<code>div > p</code>	Párrafos que son hijos directos de un div
Selector universal	<code>*</code>	Todos los elementos del documento

1.3. Propiedades

Las propiedades son lo que **define los estilos como tal**: lo que se escribe dentro del bloque del selector. Ejemplo completo con varias propiedades habituales:

```
button {
  font-size: 1.6rem;
  padding: 1rem 2rem 1rem 2rem;
  border: 1px solid var(--white-color);
  background-color: transparent;
  color: var(--white-color);
  cursor: pointer;
  display: flex;
  align-items: center;
  flex-flow: row nowrap;
}
```

★ Lo más importante

- La **cascada** decide qué regla gana en caso de conflicto: el último estilo definido, o el selector más específico.
- Existen selectores de **elemento, clase, ID, atributo, pseudoclase/pseudoelemento, combinador y universal**.
- `.clase` pegado al selector ≠ `.clase` separado por un espacio (elemento vs. descendiente).
- Si una propiedad está mal definida, CSS **no lanza ningún error**: el sitio sigue funcionando, pero el fallo es más difícil de detectar.

2. Modelo de caja

Todo elemento en CSS tiene una **caja** a su alrededor, y el comportamiento de esa caja se define mediante propiedades CSS. La diferencia se aprecia bien comparando un elemento `p` con un elemento `span`:

- Un elemento `p` añade una especie de bloque invisible que desplaza a los demás elementos y provoca un salto de línea (visualización externa **block**).
- Un elemento **span** dentro del texto de un párrafo no provoca ningún salto de línea (visualización externa **inline**).

2.1. Visualización externa: block e inline

Este comportamiento se puede cambiar a través de la propiedad `display`:

```
button {
  display: flex;
  align-items: center;
}

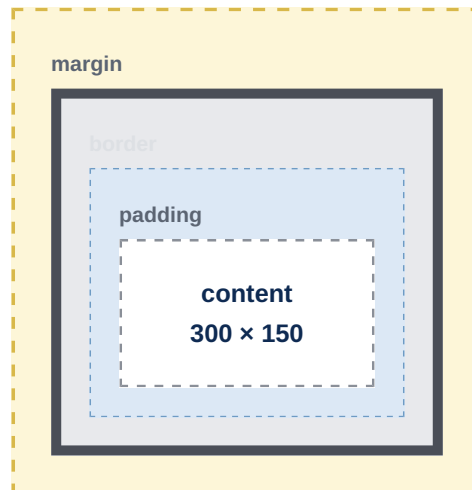
button:before {
  content: "";
  display: block;
  width: 0;
  transition: all 0.2s linear 0s;
}
```

Si a un `div` se le añade `display: flex`, la visualización **externa** sigue siendo de tipo **block**, pero la visualización **interna** pasa a ser de tipo **flex** y sigue las reglas de flexbox (se estudia en el apartado de layout).

Un caso donde el modelo de caja da más de un quebradero de cabeza es cuando queremos colocar varios elementos en fila o repartir una fila en varias columnas: por defecto, cada bloque (por ejemplo, un `div`) tiende a ocupar el 100% del ancho disponible y a provocar un salto de línea, para que el siguiente elemento se coloque debajo.

2.2. Las capas de la caja

Las cajas se dividen en varias capas, como una cebolla:



Capa	Propiedad	Función
Contenido	<code>width</code> / <code>height</code>	La capa a la que afectan directamente el ancho y el alto
Padding (relleno)	<code>padding</code>	El espacio alrededor del contenido
Borde	<code>border</code>	Envuelve el contenido y el relleno de la caja
Margen	<code>margin</code>	La capa más externa; define el espacio alrededor de todo lo demás

i El `padding` y el `margin` se confunden con facilidad, pero no son lo mismo. La propiedad `background`, por ejemplo, afecta hasta la capa del relleno, pero nunca se muestra en la capa de margen.

2.3. Box-sizing: estándar vs. alternativo

El modelo de caja **estándar** no incluye el relleno ni el borde dentro de las propiedades `width` y `height`. Por eso, si se crean 4 columnas al 25% de ancho cada una y se les añade relleno, borde o margen, el resultado final **no** ocupará el 100% como se esperaba.

Existe un modelo **alternativo** que sí incluye el relleno y el borde dentro del ancho/alto definidos, activable con la propiedad `box-sizing: border-box`:

```
html {
  box-sizing: border-box;
}
*,
*::before,
*::after {
  box-sizing: inherit;
}
```

Para inspeccionar el modelo de caja real de cualquier elemento de una página, las **DevTools** del navegador lo muestran de forma visual al seleccionar el elemento.

★ **Regla clave**

- ▶ Visualización **externa**: block (ocupa toda la fila) vs. inline (no rompe el flujo).
- ▶ Visualización **interna**: cómo se comportan los elementos dentro del propio elemento (por ejemplo, flex).
- ▶ El modelo estándar **no** incluye padding/border en width/height; `border-box` sí los incluye.
- ▶ `background` llega hasta el padding, nunca pinta el margin.

3. Variables

CSS permite definir **variables** (custom properties) para reutilizar valores de propiedades a lo largo de toda la hoja de estilos:

```
html {  
  --font-size-global: 16px;  
  --min-width: 320px;  
  
  /* COLORS */  
  --white-color: white;  
  --gold-color: rgb(201, 171, 129);  
  --dark-color: rgb(18, 20, 23);  
  
  /* LAYOUT */  
  --breakpoint-sm: 576px;  
  --breakpoint-md: 768px;  
  --breakpoint-lg: 992px;  
}
```

3.1. Sintaxis y uso

Una variable se declara añadiendo **dos guiones (--)** al comienzo del nombre y asignándole un valor de propiedad; después se consume con la función **var()**:

```
html {  
  font-size: var(--font-size-global);  
  color: var(--white-color);  
  font-family: "VR_Standard", sans-serif;  
}
```

3.2. Ámbito (scope)

El selector bajo el que se define una variable determina su **ámbito de aplicación**. Si se define bajo un selector **p**, solo estará disponible dentro de párrafos. Para que el ámbito sea global, se usa el selector **html** o la pseudoclase **:root**.

3.3. Valores de reserva (fallback)

Si se va a usar una variable que puede no existir, conviene definir una alternativa de reserva (**fallback**):

```
.tres {  
  background-color: var(  
    --my-var,  
    var(--my-background, pink)  
  );  
}
```



```
); /* rosa si --my-var y --my-background no existen */
}
```

i No se pueden **concatenar** nombres de variables como alternativa directa (por ejemplo `var(--my-var, --my-background, pink)` es inválido). Cada alternativa debe envolverse también en su propio `var()`.

3.4. Importación con @import

Es habitual tener una hoja de estilos dedicada solo a variables (por ejemplo, `variables.css`) e **importarla** desde la hoja de estilos global para que el resto de hojas puedan usarla:

```
@import url("../variables.css");
@import url("../layout.css");
@import url("../colors.css");
```

★ Lo más importante

- ▶ Se declaran con `--nombre: valor;` y se usan con `var(--nombre)`.
- ▶ El ámbito depende del selector; usa `html` o `:root` para que sean globales.
- ▶ Define siempre un **fallback** cuando exista riesgo de que la variable no esté definida.
- ▶ Con `@import` puedes centralizar variables en una hoja aparte y reutilizarlas en todo el sitio.

4. Dimensiones

A la hora de definir dimensiones para los elementos, lo más habitual es utilizar **píxeles**, aunque existen otras unidades más adecuadas según el caso:

```
h4, .header4 {
  font-size: 24px;
  font-weight: 400;
  padding: 1rem 0;
}
```

Unidad	Se calcula respecto a...	Detalle clave
px	Píxeles absolutos	No cambia aunque el usuario modifique el tamaño de letra del navegador
em	El <i>font-size</i> del elemento padre más cercano	Es relativa y acumulativa : si el padre ya escaló su fuente, el hijo escala sobre ese resultado
rem	Únicamente el <i>font-size</i> de la raíz (<code>html</code>)	De ahí su nombre, <i>Root em</i> ; ignora el <i>font-size</i> de los padres intermedios
%	El tamaño del elemento padre	Orientado sobre todo a anchos de caja flexibles (por ejemplo, 50% + 50%)
vw / vh	El tamaño del <i>viewport</i>	<i>viewport's width</i> y <i>viewport's height</i> : 100vw equivale al ancho total visible

Un ejemplo de cómo se acumula **em** frente a **rem**:

```
.guide-content {
  font-size: 1.6em; /* 1,6 × 16px = 25,6px */
}

.guide-content__container {
  font-size: 0.9em; /* 25,6px × 0,9 = 23,04px (hija de la anterior) */
}
```

Con **rem**, en cambio, ambas clases se calcularían siempre sobre el **font-size** de la raíz (16px), sin tener en cuenta el valor establecido por el padre: **.guide-content** = 16px × 1,6 y **.guide-content__container** = 16px × 0,9.

★ Lo más importante

- **em** depende del padre y se va acumulando en cascada; **rem** depende únicamente de la raíz.
- **%** es la unidad natural para repartir anchos flexibles entre columnas.

- ▶ **vw/vh** se calculan siempre sobre el tamaño del *viewport*, no sobre el elemento padre.
- ▶ Puedes fijar tu propio `font-size` base en `html` para que sirva de referencia a em/rem, en vez del que aplique el navegador (16px por defecto).

5. Posicionamiento

El posicionamiento define la **posición que toma un elemento** dentro del documento web y las reglas que permiten alterar su comportamiento estándar.

Valor	Comportamiento	Referencia de <i>top/right/bottom/left</i>
static	Comportamiento estándar de todo elemento	No aplica
relative	Igual que static, salvo que se usen las propiedades de posicionamiento	Su propia posición de origen
absolute	Sale del flujo normal y se coloca en una capa aparte	El elemento posicionado más cercano que lo contiene
fixed	Igual que absolute, pero pensado para permanecer visible	La parte visible del <i>viewport</i>
sticky	Mezcla de relative y fixed	relative hasta hacerse visible; luego se comporta como fixed

Ejemplo de `position: relative` con desplazamiento:

```
.element-moved {
  position: relative;
  top: 1rem;
  left: 1rem;
}
```

Esto desplaza el elemento **16px hacia abajo y 16px hacia la derecha**, ya que se está indicando que se deje un espacio de 1rem en la parte superior y otro en la parte izquierda respecto a su posición original.

i Piensa en `absolute` como una **modal** que se abre encima del contenido principal: no está en el flujo normal, sino en otra capa por encima. En `fixed`, un ejemplo típico es el **menú del sitio**: por mucho scroll que se haga, permanece siempre visible en la misma posición.

5.2. Z-index

La propiedad `z-index` añade la **tercera dimensión**. Con el tipo de posicionamiento y las propiedades de posicionamiento se mueve el elemento en los ejes 'x' e 'y'; `z-index` resuelve qué elemento queda por encima de otro en el eje 'z'.

★ Regla clave

- ▶ Cuando **mayor** sea el valor de `z-index`, más arriba en el eje 'z' estará el elemento.

- ▶ El valor de `z-index` es **únicamente un número**: no se utilizan unidades como px, em o rem.
- ▶ `absolute` se referencia al ancestro posicionado más cercano; `fixed`, al viewport (salvo que un padre también sea fixed).

6. Layout

El layout es lo que define la **disposición de los elementos** en el documento web. A lo largo de los años han surgido distintas técnicas para resolverlo:

CSS layout a lo largo del tiempo



6.1. Float

Hoy en día, **float** se orienta sobre todo a la **colocación flotante de imágenes** respecto del texto (izquierda, derecha), igual que en un documento de Word. Antes de la llegada de Flexbox, se usaba también junto con **position** para construir columnas — algo tedioso, ya que un elemento flotado **no ocupa espacio** de cara al contenedor, y todo lo de debajo tiende a envolverlo.

Para crear dos columnas hay que flotar una capa a la izquierda, otra a la derecha y, después, **limpiar los floats** con la propiedad **clear**:

```
.float-left { float: left; }
.float-right { float: right; }
.wrapper::after {
  content: "";
  display: block;
  clear: both;
}
```

Este truco (*clearfix*) limpia todos los floats que existan justo después del elemento **wrapper**.

6.2. Flexbox

Flexbox (2012) elimina los problemas ocasionados por **float** añadiendo un nuevo valor a la propiedad **display**, autocontendiendo el flotado de los elementos (ítems) dentro de un bloque:

```
.row {
  display: flex;
  flex-wrap: wrap;
  margin: 0 auto;
  width: 100%;
}
```

Propiedad	Qué permite
<code>flex-direction</code>	Distribuir en fila o columna, incluso invertidas al orden natural
<code>order</code>	Decidir el orden de un ítem concreto
<code>flex-grow</code>	Indicar la proporción de espacio que ocupa cada ítem
<code>flex-wrap</code>	Adaptar los ítems a una única línea o dejar que salten de línea
<code>justify-content</code> / <code>align-items</code> / <code>align-content</code>	Distribuir el contenido horizontal y verticalmente; <code>align-self</code> lo hace para un ítem en concreto

6.3. Grid

CSS Grid (2017) resuelve algunas complicaciones de Flexbox cuando la estructuración del contenido es más compleja: permite definir una **cuadrícula** completa desde el elemento contenedor.

Se activa con `display: grid` (o `inline-grid`). Las propiedades principales son:

- `grid-template-columns`, `grid-template-rows` y `grid-template-areas`: definen cantidad y tamaño de columnas/filas, con unidades como `px`, `%`, `fr` (fracción de tamaño), `auto`, o funciones como `repeat()`.
- `column-gap`, `row-gap` y `gap`: separaciones entre columnas y filas.
- `grid-column-*`: para colocar ítems específicos en columnas y filas concretas.
- Las mismas propiedades de alineación ya vistas en Flexbox también aplican en Grid.

★ Cómo elegir entre Float, Flexbox y Grid

- **Float** → solo para el flotado puntual de imágenes junto al texto; ya no se recomienda para construir columnas.
- **Flexbox** → distribuciones en **una dimensión** (una fila o una columna): barras de navegación, tarjetas, listas.
- **Grid** → distribuciones en **dos dimensiones** (filas y columnas a la vez): la cuadrícula completa de una página.

7. ¿Qué hemos aprendido?

Ya sabemos cómo dar **estructura y estilo** a un sitio web con cierto nivel de complejidad: desde elegir el selector correcto y entender la cascada, hasta dominar el modelo de caja, las variables, las unidades de medida, el posicionamiento y las principales técnicas de layout.

No olvides aplicar correctamente unidades, variables, propiedades y selectores, para que tanto el **entendimiento** como el **mantenimiento** de tus estilos sea una tarea sencilla.

★ Resumen rápido de todo el bloque

- ▶ **Selectores:** la cascada resuelve conflictos según especificidad y orden.
- ▶ **Modelo de caja:** content → padding → border → margin, como una cebolla.
- ▶ **Variables:** `--nombre` + `var()`, con ámbito y fallback.
- ▶ **Dimensiones:** px es absoluto; em/rem/% son relativos; vw/vh dependen del viewport.
- ▶ **Posicionamiento:** static, relative, absolute, fixed y sticky, resueltos en el eje 'z' con z-index.
- ▶ **Layout:** Flexbox para una dimensión, Grid para dos; float, solo para imágenes.

Practica, practica, practica ¡y practica!
Solo así te convertirás en un maestro de la maquetación.
